

CODE OF YOUR ABSTRACT

TIME-SERIES GHI FORECASTING FOR THE UTILIZATION OF PV SYSTEMS USING THE LONG-SHORT-TERM MEMORY NEURAL NETWORK MODEL

Phi Anh Tran^{1,*}, Phong Anh Luong¹, Xuan-Truong Nguyen¹

Author Affiliations

¹¹*Department of Applied Engineering and Technology, University of Science and Technology of Hanoi*

*Corresponding author: phita.bi12-343@st.usth.edu.vn

Abstract. With the rising interest in the development and deployment of photovoltaic systems as a promising renewable source of energy, the demand for these networks to be widely adopted has consequently grown, along with the need to fulfill specific essential requirements needed for such network models to be practically integrated. The ability to predict the magnitude in which solar irradiance is distributed within a general area is one of those necessities, playing a significant role in managing and making informed decisions in designing and enhancing the power efficiency of a photovoltaic power grid. Although numerous methods have been proposed with the accomplishment of this task in mind, this paper will examine one specific numerical, data-driven technique of forecasting solar irradiance, utilizing the long-short-term memory model of neural networking and built upon local weather data collected via online sources.

Keywords: solar irradiance, PV system, PV modules, renewable energy, weather forecasting, neural network

1. Introduction

In the age of the imminent energy crisis and rising global warming, the utilization of sustainable energy sources has been of focal interest within the energy community. One primary alternative source of renewable power that has caught major attention from scientific and engineering communities at large is solar energy, more specifically the integration of photovoltaic solar panels into power systems, whether it be of minimal or substantial scale. Due to the unsystematic nature of solar power, managing a PV grid proves to be more problematic when these unpredictabilities are taken into account. There are methods to remedy these intermittencies and the fluctuations experienced in the conversion of solar energy, however, for these techniques to be sufficiently practiced, the forecasting of solar irradiance is an essential requirement. Currently, there has been a variety of procedures being developed and implemented with the purpose of forecasting solar irradiance, these methods can be generalized into four commonly seen approaches: data-driven, image-based, numerical weather prediction (NWP) models, and hybrid. Data-driven methods work

on the basic principle of predicting future irradiance values based on pre-existing historical data, image-based forecasting utilizes the images taken from satellite or shadow cameras and numerical weather prediction integrates computer programs into the mix. The method being examined in this study will be that of a hybrid between the data-driven approach and NWP models, specifically, this procedure will consist of feeding the numerical program historical data taken from a reliable online database and running it through a time series prediction model to get the forecasting values required to operate a solar system. The method of predicting solar irradiance demonstrated in this study is made possible by the utilization of a type of neural network called a recurrent neural network (RNN), specifically Long short-term memory (LSTM) networks [1], which are a class of RNN designed for problems involving sequence predictions. In a typical recurrent neural network, the previous step's output is fed into the input of the upcoming step, the invention of LSTM networks dismisses the issue of "long-term reliance" in a classic RNN, which is where neural networks are unable to process and predict

data held within long-term memory, causing the gradients in back propagated networks to either increase to infinity or decrease to zero [2].

2. Methodology

a) The Structure of an LSTM network

The architecture of an LSTM network consists of four neural network layers [3, 4, 5] working simultaneously to produce a cell output and a cell state, which can then be passed onto the hidden layers. Gates were introduced as a means to restrict the amount of information that is fed into the cell, determining which output information is required by the upcoming cell and which can be discarded by assigning the values in the range between 0 to 1, where it is either associated with 'reject all' or 'include all', respectively. The main three gates of an LSTM model can be categorized as follows:

- **Forget Gate:** Initially, the LSTM program decides which information to feed into the next step and which to dispose of. Two inputs are fed into the gate, then proceeded to be multiplied and added with the model's weights and bias, respectively. The resultant output is then transferred through an activation function layer that generates a binary result between 0 and 1, where 1 represents the retainment of the information and 0 means its removal.
- **Input Gate:** This gate's function is to determine how much of the cell state memory will be modified. The information will be first regulated by the Sigmoid activation function and then filtered similarly to what happened in the forget gate. Furthermore, the TanH function may be involved in giving the data weights that represent their significance on a scale from -1 to 1. Finally, useful information can be obtained by multiplying both the regulated and significant values.
- **Output Gate:** The final act of filtering the most useful data from the cell state and presenting it as output can be accomplished by this gate. A Sigmoid activation layer is first run to regulate the section which will be turned into the final output. This portion is then multiplied by the cell state after passing

it through the TanH function to convert the values into a range between -1 and 1.

The problem we will look at in this study is the solar irradiance (radiation) prediction problem. In which, given the date and measured value of a certain day, the goal is to predict the changes in said values after a predetermined amount of time into the future. In this experiment, we will build a program that, by processing the input data, can produce rough estimations that we can later compare to the true values. The provided data ranges within a period of 2 years, from September of 2021 to September of 2023, with 730 observations.

b) The weather information

With the increasing unpredictability of climate changes, the need for weather records and forecasting has become essential in the prevention of natural disasters. This necessity was further realized and made exceptionally accessible with the aid of free online resources. In this study, we will be using the weather history data provided by the Visual Crossing Weather database [6].

The dataset for the city of Hanoi in Vietnam is available with daily records within spans of days to years and comprises over 30 features. But for simplicity's sake, we will only be looking at the reported values for solar radiation within the last 2 years and use the previous year's records as relevant training data for the program.

After importing the necessary libraries, we download and store the datasheet locally as a comma-separated values (CSV) file. It is also possible to extract the zip content from the download link into a CSV file using the zipfile library but to simplify the process, we will be working with locally stored data. After removing irrelevant information, we allow the program to "read" the dataset by linking the content of the file with the library Pandas.

In order to see the first 5 values from the list we just imported, we use the `df.head()` function, or we can plot the datasheet using Matplotlib. The figure size in Pandas is defined by width and height in units of inches, in this example we use a figure size of 12x6 inches (Figure 1a).

After receiving the figure and having a general idea of how the values change over time, we investigate the data further with the help of Time Series Decomposition, using seasonal

decomposition from Statmodel. This will give us the trend and the seasonal components of the dataset, as seen in Figure 1b.

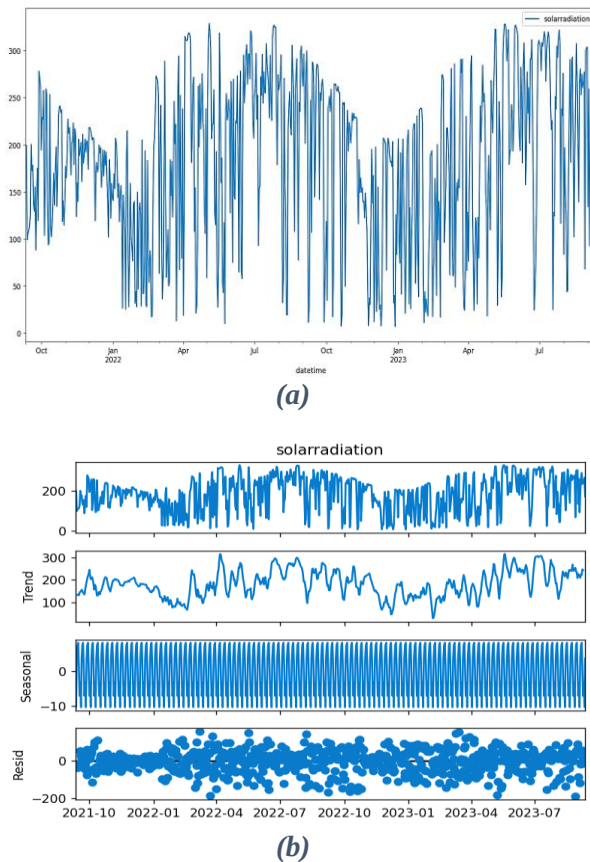


Figure 1. (a) the graphical representation of the dataset (b) The seasonal decomposition of the data frame

c) Data preprocessing

Before the preprocessing of our data can begin, we first must create a Pandas frame for the program to work on. This can be achieved using `pd.DataFrame` and specifying the name as well as the columns necessary to construct the data frame, we can discard the `DateTime` column as it is not relevant to the process.

For this prediction model, we will only be using the first 365 days' worth of measurements, therefore it is important to specify the number of data to build upon in a new data frame, one which we will later use to construct our model. It is possible to observe the shape of our new frame using the `print(df.shape)` command.

The null values or the number of missing values can be examined and then removed. In this instance, there are no unknowns within the set, the shape of which remained unchanged. The

relevant values are then stored in an array and turned vertically.

We initialize the normalization of the data by importing the `MinMaxScaler` function. The purpose of this is the conversion of the measured data from their original values into that on a common scale of 0 to 1, allowing better training stability and overall performance of the model by simplifying and speeding up the learning process [7, 8].

Figure 2 showcases the initial 5 values of the array before and after data normalization:

```
original values: [[186.5]
 [199.2]
 [ 99. ]
 [106.3]
 [109.2]]
re-scaled values: [[0.55363864]
 [0.59347553]
 [0.27917189]
 [0.30207026]
 [0.31116688]]
```

Figure 2. Normalization of the first 5 values from the data

d) Training and testing dataset

After our data normalization, we proceed to map out the basic principles on which our prediction model will be trained. The fundamentals of this process are as follows, the input gate receives the first 20 values, and the output gate will return the next value in sequence to the values that the input was presented with. Subsequently, the next collection of input data will discard the first value of the previous set and instead add the last "prediction" value. As previously, the output of this one will comprise only a single number that is succeeding to that of the previous output value. After this process is finished, the next important step is to reshape the two input and output lists that were created into arrays that the Numpy library can identify and work with. This can be accomplished by utilizing the `np.asanarray ()` function in NumPy.

For the testing and training section, we elect 65% of the length of the original data to work with, meaning we are taking 65% of 365 days of measured values which equates to 237.25 rounded up into 237. These 237 numbers are then split into two sets, of which the first half of inputs and outputs go into the "x_train" and "y_train" arrays, while the latter half resides within "x_test" and "y_test", we then move on to the next section.

e) Building the LSTM model

For the program to be correctly replicable in later experiments, we must adjust the initial random seed that would later become the weight of our LSTM network. The effects of these seeds can be modified within the TensorFlow library by generating a specific value for the initial weight utilizing the Python Random Module [9]. The number chosen in this study is 343, this was selected from a list of arbitrary numbers based on how much error each value produced when run accordingly with the computer model.

Prior to the construction of our LSTM model, we imported all the necessary libraries. This includes matplotlib, seaborn (for creating statistical graphics), and keras tensorflow layers as well as models.

Model building is easily the most crucial part of this entire program, this process establishes the foundation upon which the LSTM predictions will be made. The proceeding codes are based on the sequential model, specifically the stacking of multiple LSTM layers where each layer contains multiple memory cells [10, 11]. The purpose of increasing the depth of our model is to heighten the learning accuracy and decrease the amounts of neurons needed, consequently, greatly reducing the training time required by the LSTM program. There are 95 neurons within each layer, this number was shown to produce the most desirable results for the overall model.

The model summary is shown in Figure 3.

```

Model: "sequential_1"
-----
Layer (type)                Output Shape              Param #
-----
lstm_3 (LSTM)                (None, 20, 95)           36860
lstm_4 (LSTM)                (None, 20, 95)           72580
lstm_5 (LSTM)                (None, 95)               72580
dense_1 (Dense)              (None, 1)                96
-----
Total params: 182116 (711.39 KB)
Trainable params: 182116 (711.39 KB)
Non-trainable params: 0 (0.00 Byte)
-----
None
    
```

Figure 3. LSTM model summary

With all model parameters readily established, the remaining step is validating the model by fitting it onto the training set. We define the number of epochs as 100, meaning the dataset will be passed completely through the algorithm 100 times, and the data's model internal

parameters are updated after each completed epoch [12]. This number was selected after numerous tests with the purpose of determining the most sufficient amount of epochs, allowing the program to run continuously until an acceptable output error is achieved. Lastly, we set verbose equals to 1, as such the program will display the training process in the form of an animated progress bar. The model training was completed from start to finish within an impressively short amount of time, approximately 23 seconds. For reference, a single-layered LSTM model with the same number of neurons and epochs would require roughly 45 seconds to train, while producing substantially less accurate results.

f) Prediction and model evaluation

An examination of the validity of the model is now required. We start by considering the training values produced by the model based on the given data in comparison to the true measurements. This can be achieved by applying the model prediction function to the training input set, to do this we call the name of the array after the function and specify the index of the value we want prediction of. For this example, we will be looking at the fifth index value of the array (Figure 4a). As we can see, the model's prediction was relatively close to the true value, we can confirm that the model worked as expected and move on to examining its effects on the testing set. Similarly, the input set is applied to the function, and the output set is called in for reference to the accuracy of the predictions. The result for the prediction of the testing set's second index is shown below in Figure 4b.

```

Predicted Value [0.45337617]
Expected value [0.45671267]
    
```

(a)

```

Predicted Value: 0.6472112
Expected Value: 0.689146800501882
    
```

(b)

Figure 4. (a) Training set's prediction value
(b) Testing set's prediction value

After it has been verified that the model is functioning appropriately, it is time to develop and observe the full extent of the prediction capacity on the entirety of the testing dataset. We start by creating a new Pandas integrated data frame for all

the predicted values, construct another column under the name “TrueValues” containing the data of the “y_test” array, and then finally rename the zeroth column of the data frame consisting of the predicted values as “Predictions”. The entire prediction data frame will be presented similarly as shown in Figure 5.

```
new prediction dataframe:
      Predictions  TrueValues
0      0.538502    0.766939
1      0.662250    0.678795
2      0.647211    0.689147
3      0.649617    0.745922
4      0.674689    0.703262
..          ...          ...
```

Figure 5. Completed prediction data frame

Lastly, for further diagnosis and analysis of the statistical features, a graphical plot of the experimental values is compared to that of the predicted values. For this, the line plot function from the Seaborn library is applied to the data frame. The finished graphical representation of our model’s predictions is presented in Figure 6. The yellow line represents the true values and the predicted values are shown via the blue line.

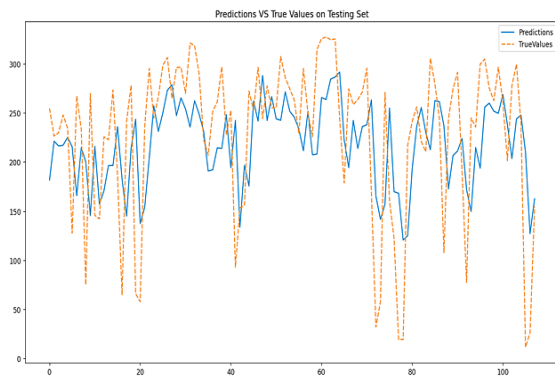


Figure 6. Comparison between measured and predicted values

g) Calculating the root-mean-square error

We further investigate the LSTM’s model performance and its accuracy by calculating the average difference between the predicted values and the true measurements, this task can be performed by importing the mean_square_error function from the sklearn library and introducing it to the values of the two relevant datasets. The resultant root-mean-square deviation for these specific settings of the prediction model is

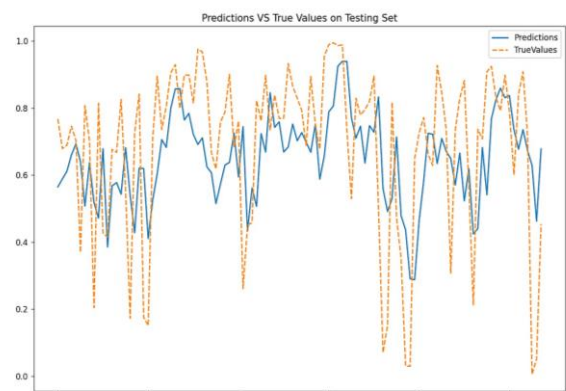
approximately 21%, a promising result for an increasingly unpredictable and capricious form of experimental data.

3. Results and discussion

3.1. Results analysis

For the remainder of this study, we further analyze the results yielded from this experiment. By recognizing overall patterns in the data frame, the program makes calculations via the algorithm established in the codes and provides numerical predictions based on the input we fed them. However, there exist a few noticeable deviations from our standard output, especially in places where there are sharp changes in the radiation measurements due to unpredictable weather conditions, this increases the difficulty of forecasting. These deviations can be attributed to many reasons such as over-fitting inside of the system or an inadequate amount of input data. These can be minimized by providing the model with a larger collection of input, resulting in better training and testing of the system, the trade-off will be that of processing time since there is a larger number of data, prolonging the epoch training period.

The complexity of the datasheet also contributes greatly to the efficiency of the prediction model. Provided with the same set of data, we can alter the number of neurons and epochs required to run the program to further analyze the best-suited combination for the neural network. Below are two of our other attempts at producing more accurate results by changing the specifications of the model, each of these provides us with reasonably close predictions (Figure 7).



(a)

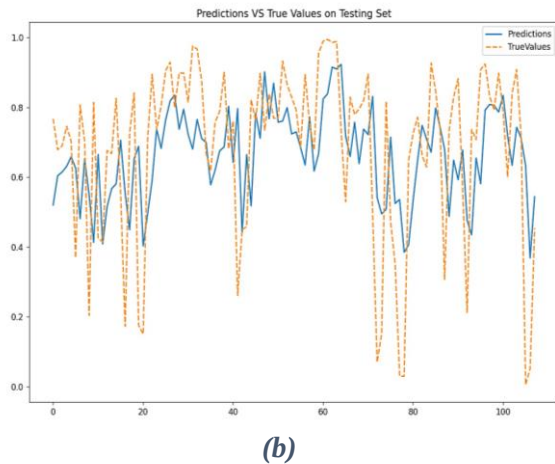


Figure 7. Prediction results after adjustments were made to the model's specifications (a) 125 epochs with 100 neurons (b) 100 epochs with 90 neurons

To demonstrate how the model performs with simpler measurements, we provide the program with data on the hourly measurements of solar irradiance and the daily temperatures of Hanoi in place of daily average solar radiation values. As expected, the LSTM network performance improved significantly, with all the specificities unchanged, reducing the root mean square error to a mere 10% and 5%, respectively (Figure 8, Figure 9). The provided data for the daily average temperatures are given as their post-normalization values and can be converted back to their original sizes by conducting further minor programming. This test with these two datasets confirms the validity of the network model and the difficulty in predicting data that are considerably more erratic in nature.

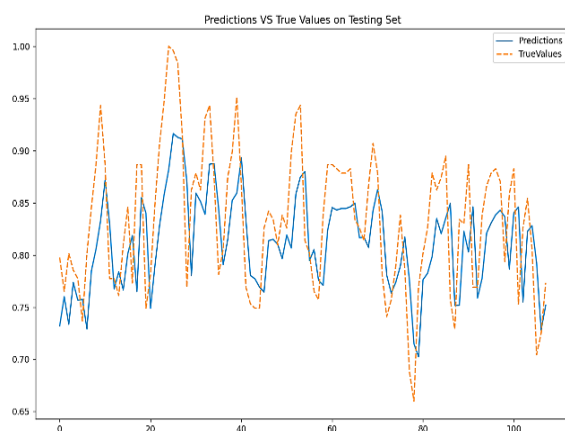


Figure 8. Predictions vs True values based on daily temperature measurements

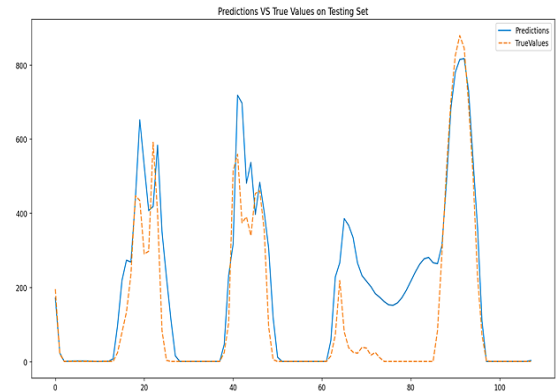


Figure 9. Predictions vs True values based on hourly irradiance measurements

Undoubtedly, even though the results from this prototype program are relatively satisfactory, there remains much room for more improvements to be made. We believe that given what has been accomplished in this experiment, provided with the right resources and an adequate amount of time, this prediction model may be able to fulfill its full potential in the foreseeable future.

3.2. Further discussion

With a working solar irradiance forecasting model established, there are numerous ways it can be implemented into practical applications. With a solar radiation forecasting system, we can record real-time data and feed it into the program, with one of the advantages of having a short calculation time, it excels in predicting short-term output, which then can be used to micro-manage devices within an embedded system. One of these applications can be seen in auto-adjusted rotating solar panels.

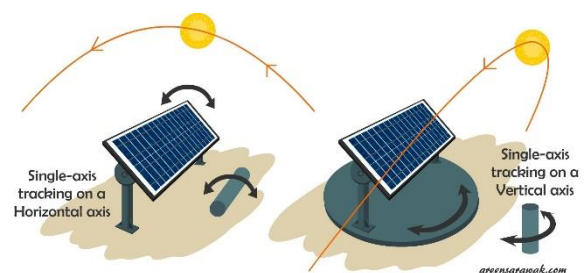


Figure 10. Auto-adjustable solar panels

By having an accurate and fast prediction of the solar output within the upcoming hours, we can design rotating solar panels that will respond to the environment according to the prediction it is receiving (Figure 10), increasing efficiency and

heightening the panel's net energy output. For instance, if the surface temperature is predicted to intensify within the receiving range, the panels will rotate automatically in response to the newly acquired information, protecting the panels from overheating all the while keeping up the productivity of the system.

This class of fast-responding predicting systems works best in interconnecting embedded systems with many small sensors. Another noteworthy application is the utilization of smart homes, currently one of the more highly appealing, rising markets. With this model implemented, the user can better manage the output of PV systems integrated into their living space, and control the balance of energy stored within the structure more accurately, decreasing the risk of shortage or fires due to overloading of the system.

4. Conclusion

With the newly approved National Power Development Plan VIII (PDP8), the strive towards green energy and renewable power sources has become increasingly relevant. Thus escalating the demand for solar power to be sufficiently integrated. This can be achieved via the development of transmission hardware as well as intelligent software that allows users to control and navigate the production of electricity. This justified the relevance of a solar radiation forecasting system. The work reported in this paper is one piece of a larger puzzle that together forms a complete picture of the power potential of solar energy. The LSTM neural network model helped achieve the level of complexity and accuracy in predictions that were impractical in less-developed scientific computations of the past, yet more accessible for the general public to research and explore. The wide range of applications coupled with its accessibility makes this forecasting model exceptionally effective, especially when data analysis and deep/machine learning are involved. The automation of photovoltaic panels and other smart devices as well as industrial applications are also in abundance when it comes to LSTM recurrent neural networks. Although the results shown in this paper are moderate, we hope they illustrate a fraction of the potential that deep learning holds along with the many ways it could improve our current approach to energy production, a topic which we would certainly enjoy revisiting in the future.

References

- [1] Aditiano, "Understanding of LSTM networks", *Geeksforgeeks*, June. 2023.
<https://www.geeksforgeeks.org/understanding-of-lstm-networks/>
- [2] Infolks Group, "Recurrent Neural Network and Long Term Dependencies", *Medium*, July 14. 2019.
<https://infolksgroup.medium.com/recurrent-neural-network-and-long-term-dependencies-e21773defd92#:~:text=LSTM%20%E2%80%94%20the%20Solution%20to%20Long,based%20on%20the%20previous%20words.>
- [3] Nick McCullum, "Deep Learning Neural Networks Explained in Plain English", *FreeCodeCamp*, June. 2020.
<https://www.freecodecamp.org/news/deep-learning-neural-networks-explained-in-plain-english/#:~:text=Neurons%20in%20deep%20learning%20models,layer%20of%20the%20neural%20net.>
- [4] "Weather forecast using LSTM networks", *TutorialsPoint*, accessed September. 2023.
https://www.tutorialspoint.com/keras/keras_time_series_prediction_using_lstm_rnn.htm#
- [5] Adil Lheureux, "Weather forecast using LSTM networks", *Paperspace*, January. 2023, (Digital Ocean, NYC).
<https://blog.paperspace.com/weather-forecast-using-lstm-networks/>
- [6] Visual Crossing Corporation, Virginia, USA, 2023.
<https://www.visualcrossing.com/>
- [7] Jason Brownlee, "Time Series Prediction with LSTM Recurrent Neural Networks in Python with Keras", *Machine Learning Mastery*, 7 August.2022.
<https://machinelearningmastery.com/time-series-prediction-lstm-recurrent-neural-networks-python-keras/>
- [8] D. S. Kumar, G. M. Yagli, M. Kashyap and D. Srinivasan, "Solar irradiance resource and forecasting: a comprehensive review", *The Institution of Engineering and Technology*, July. 2020, (John Wiley & Sons, Inc, 2023).
<https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/iet-rpg.2019.1227>
- [9] Nikhi Lagggarwal, "Python Random Module", *Geeksforgeeks*, April. 2023.
<https://www.geeksforgeeks.org/python-random-module/>
- [10] François Chollet, "The Sequential model", *Keras*, April. 2020.
https://keras.io/guides/sequential_model/
- [11] Jason Brownlee, "Stacked Long Short-Term Memory Networks", *Machine Learning Mastery*, August. 2019.
<https://machinelearningmastery.com/stacked-long-short-term-memory-networks/>
- [12] Jason Brownlee, "How to Tune LSTM Hyperparameters with Keras for Time Series Forecasting", *Machine Learning Mastery*, August. 2020.
<https://machinelearningmastery.com/tune-lstm-hyperparameters-keras-time-series-forecasting/>